

Numerical Analysis Final HW

nijatabbasov06

December 2024

Table of Contents

1	Problem 1.4	2
1.1	Part 9.1	2
1.1.1	Part A	6
1.1.2	Part B	7
1.1.3	Extinction	9
1.2	Part 9.2	10
2	Problem 1.2	11
2.0.1	Problem 5.23	11
2.0.2	Problem 5.24	11
2.0.3	Extra	12
3	Problem 1.3	14
4	Problem 1.5	15
4.1	Reference	16

1 Problem 1.4

1.1 Part 9.1

The followings are the plots for population as the time increases. The initial values for predator and prey is 10 and 100, respectively.

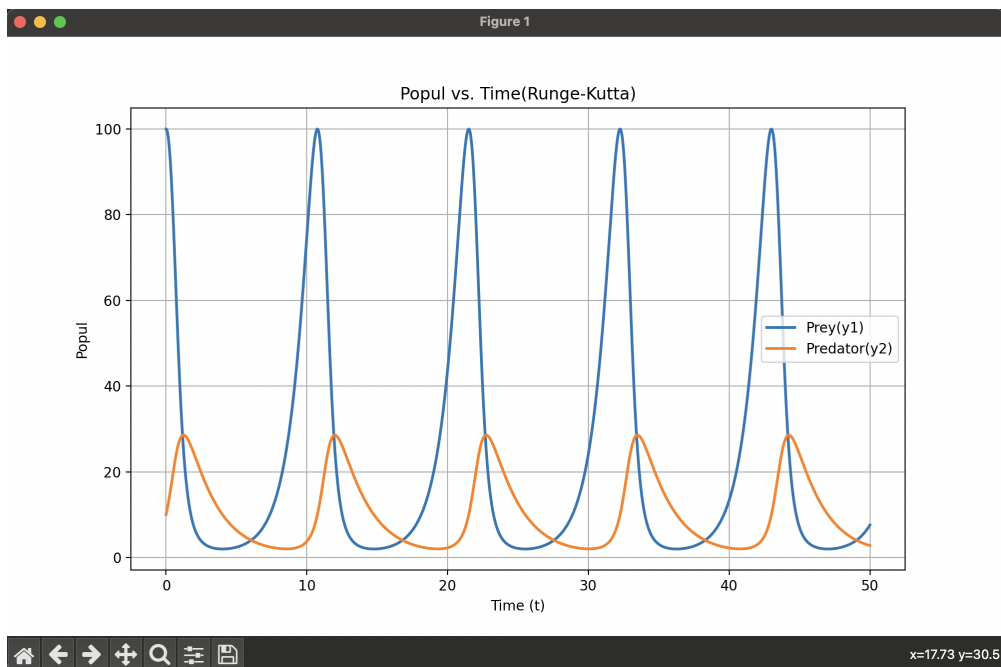


Figure 1: Population vs Time Volterra

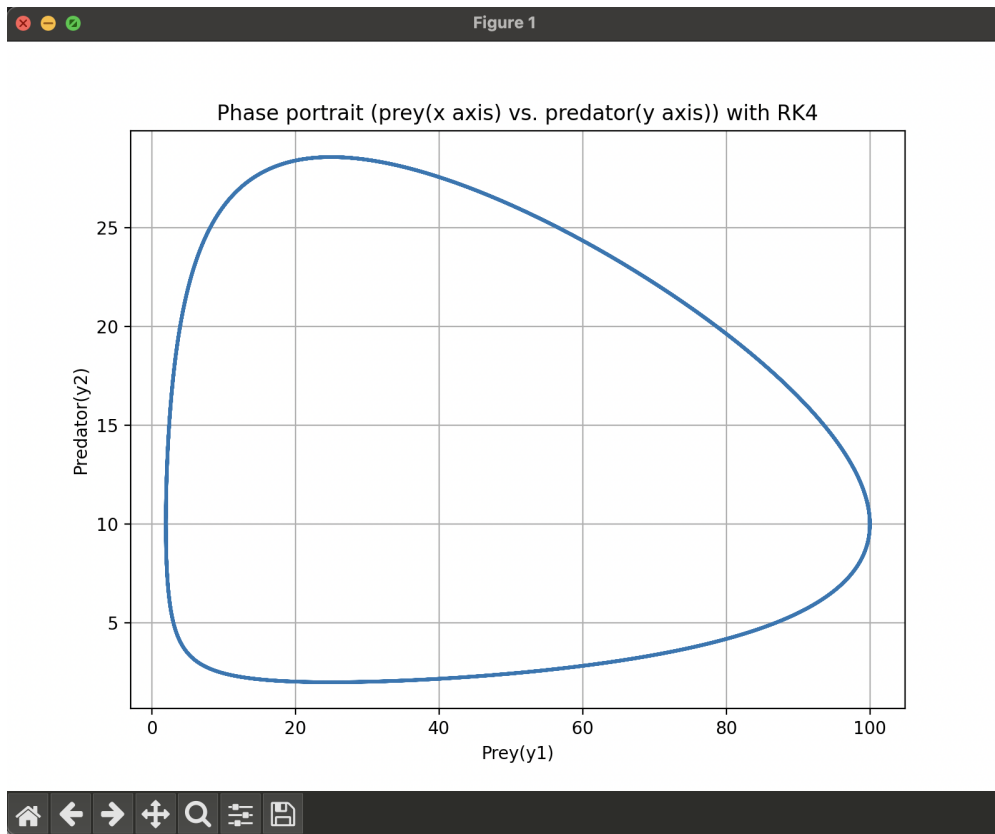


Figure 2: Predator vs Prey Volterra

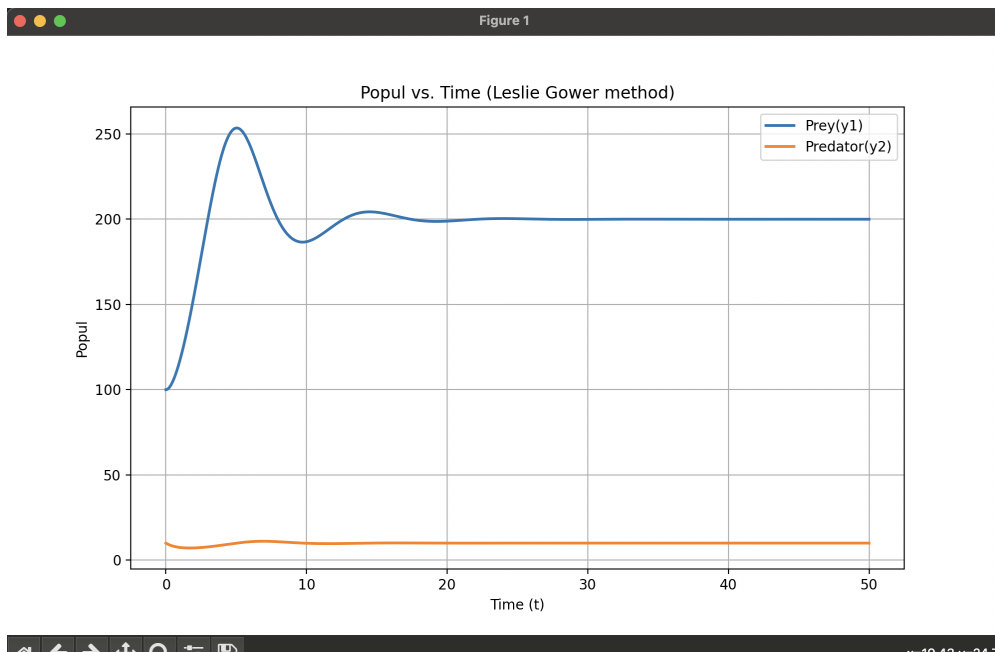


Figure 3: Population Vs Time Leslie

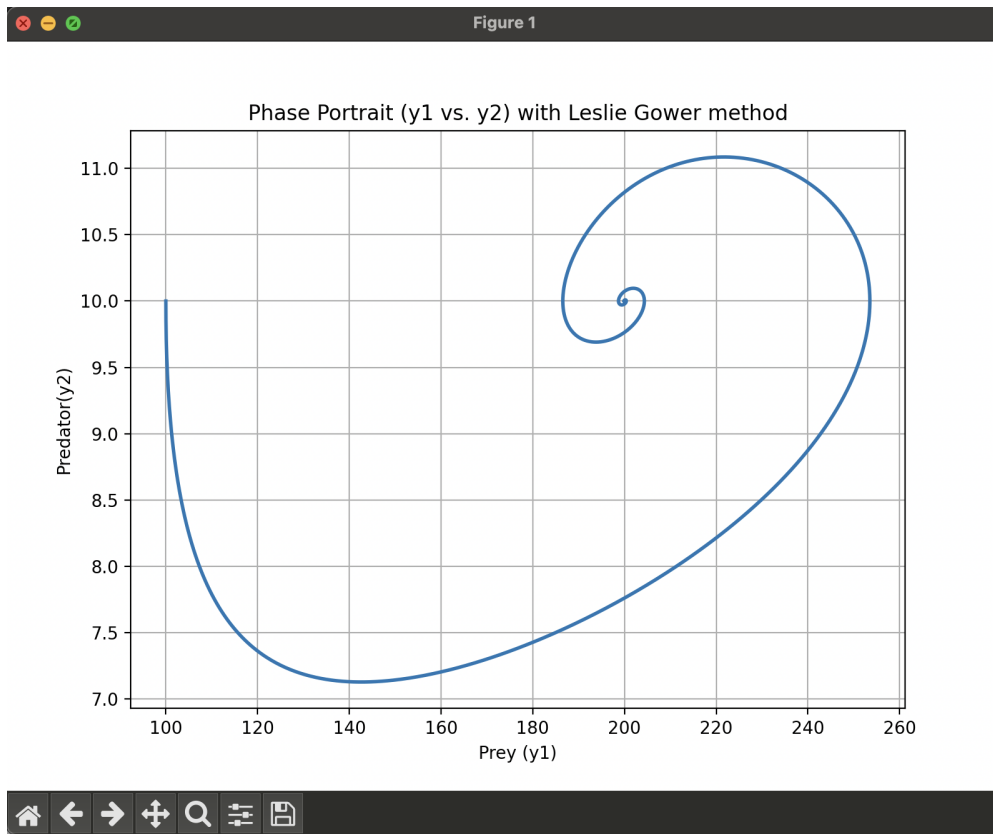
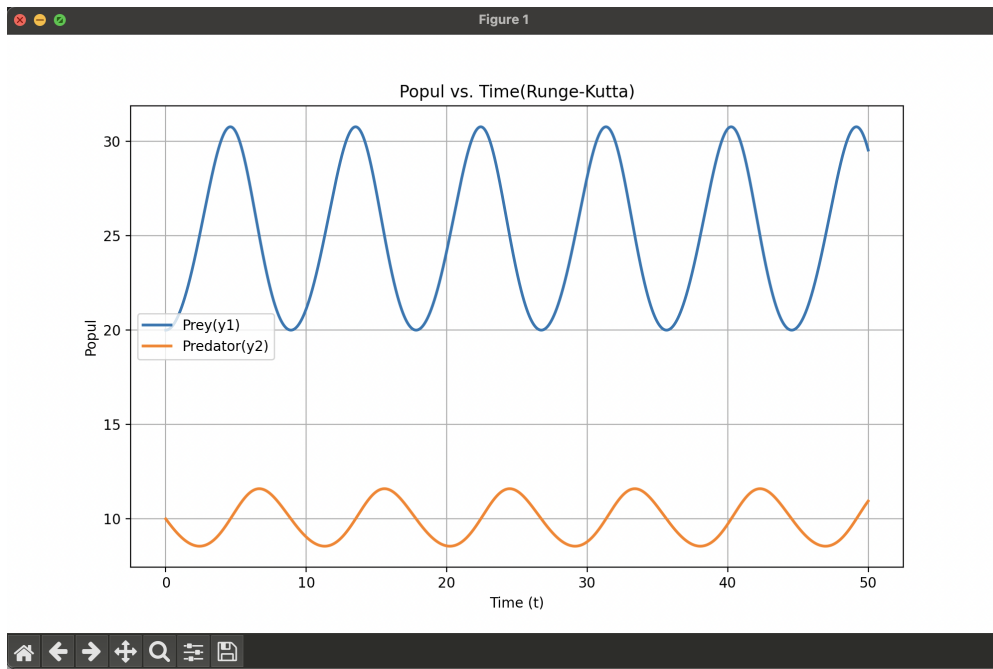
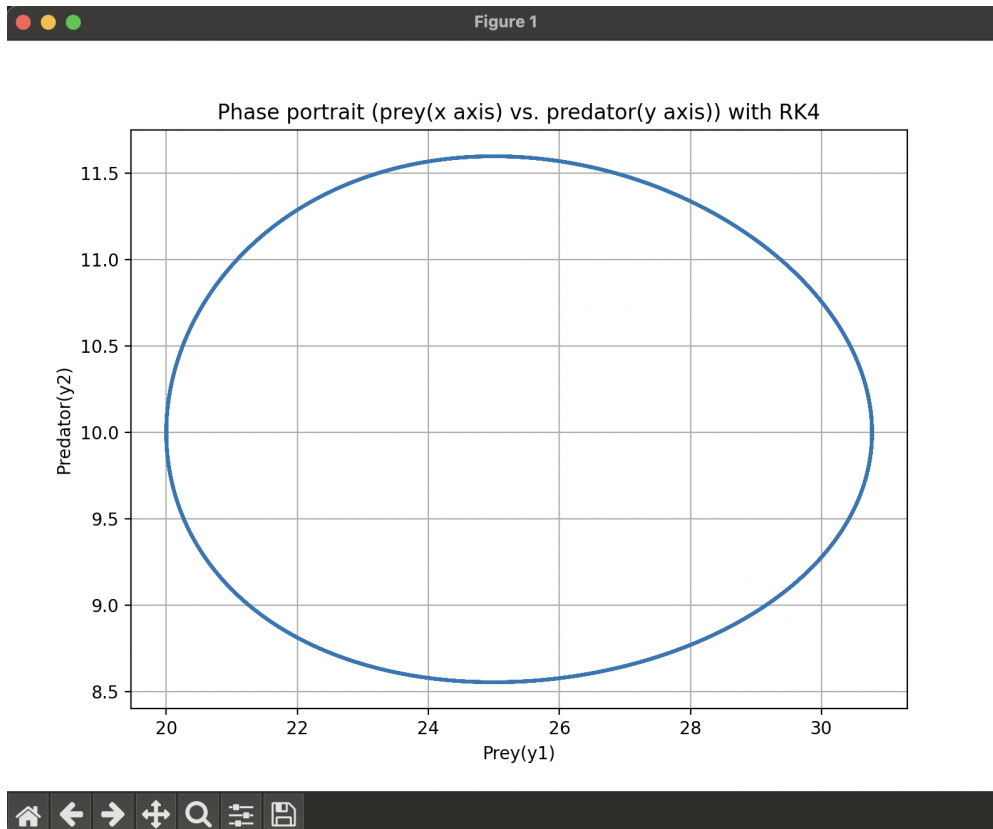


Figure 4: Predator vs Prey Leslie Gower

When the starting y values are 20 and 10:

Figure 5: Population vs Time $y = [20, 10]$ Figure 6: Predator vs Prey $y = [20, 10]$

The first two graphs represent the graphs for Volterra model and 3rd and 4th two graphs represent Leslie model when initial points are 100 and 10.

We can see that for the 3rd and 4th figure, there are some oscillations, and then they reach stability points. Those stability points corresponds to derivative of y being zero.

Last two figure represent Volterra model for initial points 20 and 10. From the last two figures, we can see that if we would say the initial values for y would be 25 and 10 then the oscillation amplitudes would become zero, and the graphs would be straight lines.

1.1.1 Part A

The stability points are 25 and 10 We can see this by equation the 2 equations for derivatives of y_1 and y_2 in the Lotka Volterra model to zero to find $y_1 = \frac{\alpha_2}{\beta_2}$ and $y_2 = \frac{\alpha_1}{\beta_1}$. Now, let's test this by giving initial values:

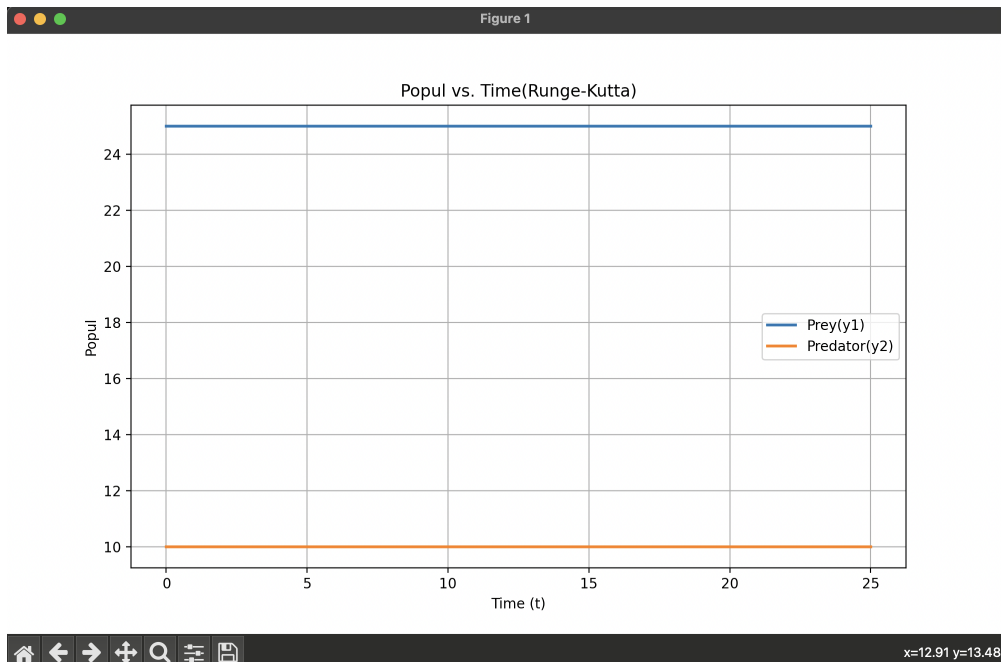


Figure 7: Stability inputs

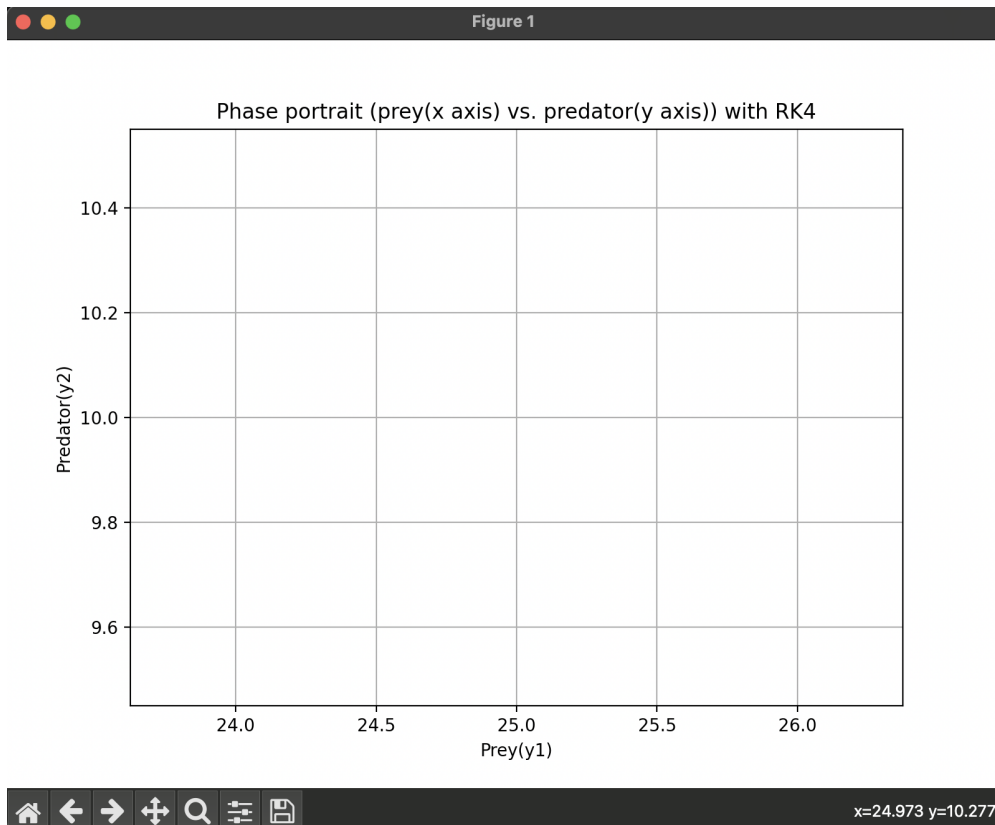


Figure 8: Stability Inputs Predator vs Prey

We cannot see anything in Figure 4 as expected. The graph only consists of one point so it is impossible to see anything.

1.1.2 Part B

Let's have a look at the graphs for Leslie model - Figure 1 and 2. We can see that at the end, the values for y_1 and y_2 converges to 200 and 10. This explains that if the initial values for y_1 and y_2 would be exactly them, 200 and 10, the values for the y_2 and y_1 would not change. Because if we treat those two equations as a system with input y_1 and y_2 and the output as the derivative of the inputs. We can see that the system would be time invariant. This means that the output of the system would give the same outputs if the inputs are same not depending on the time the inputs were given to the system. This means that, if we were to give the

inputs y_1 and y_2 as 200 and 10 to the system at time 0, the derivatives of y_1 and y_2 would be the derivatives of the 2 functions in figure 1 when $t \leq 20$, because the values for y_1 and y_2 are 200 and 10 at that time range. We can say this because for our system it does not matter at what time the y_1 and y_2 are 200 and 10. So, we can confidently say that if the inputs were to be 200 and 10, then the values of y_1 and y_2 would not change. Using the same method, in Part A, we can see from the graph that stable points are $y_1(0) = 200$ and $y_2(0) = 10$. And we can also find these values by equation the two equations for $\frac{dy}{dx}$ to zero in Leslie model to find $y_1 = \frac{\beta_2 y_2}{\alpha_2}$ and $y_2 = \frac{\alpha_1}{\beta_1}$.

Now we test this by changing our initial values accordingly:

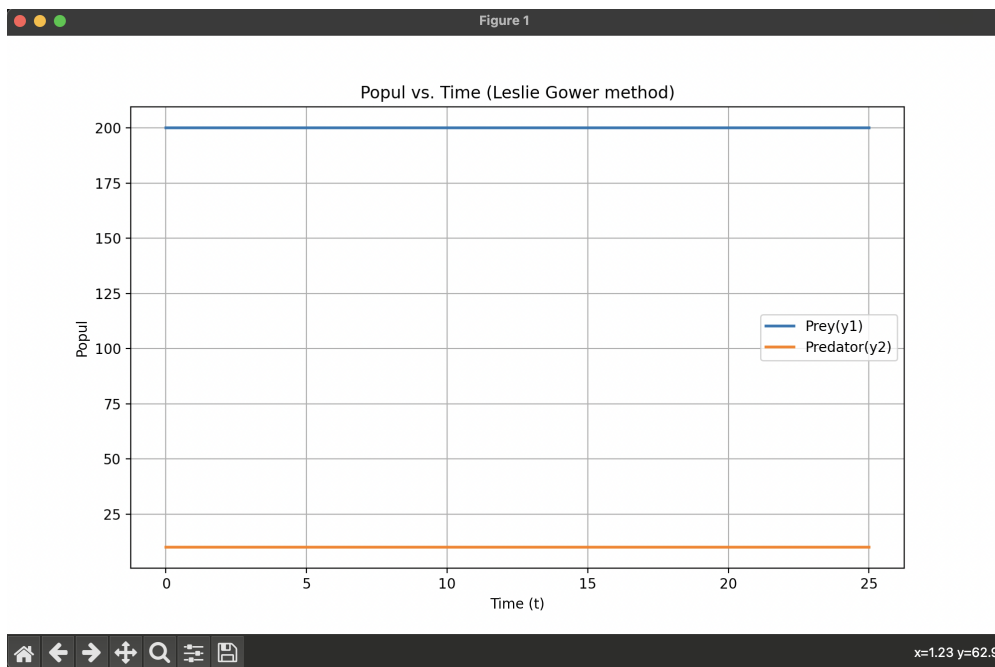


Figure 9: Stability inputs part B

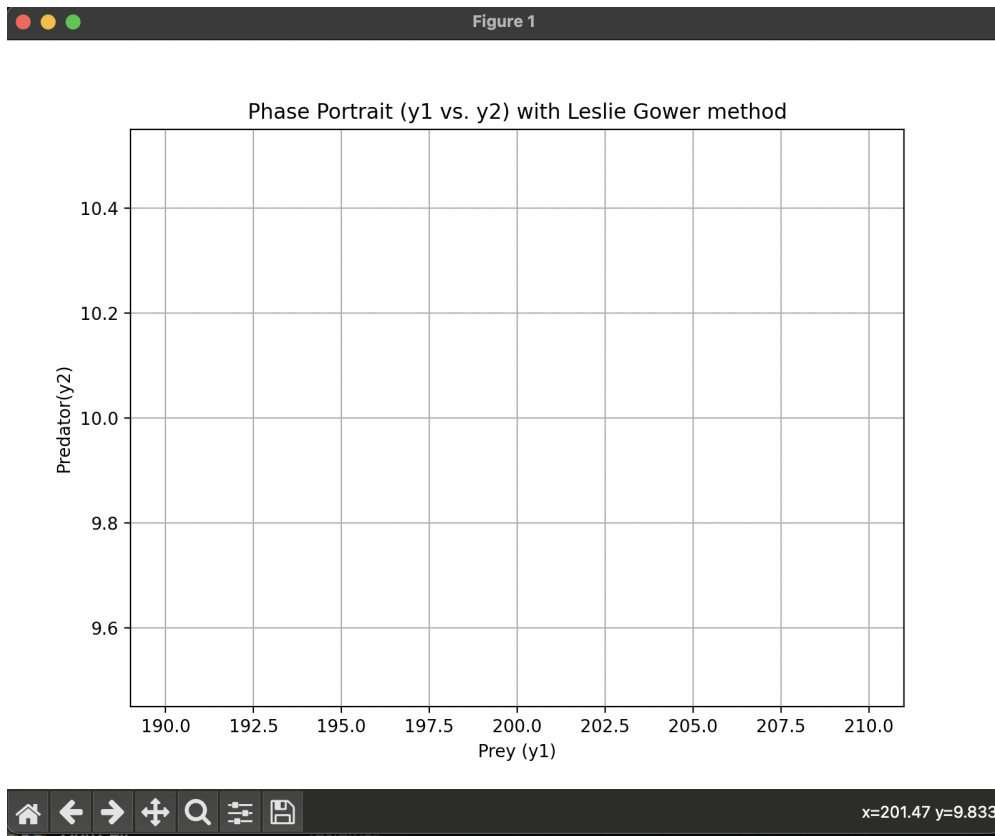


Figure 10: Stability Inputs Predator vs Prey Part B

1.1.3 Extinction

There is no nonzero initial values which causes extinction for Volterra model, the predator and prey numbers can only reach zero but then they will become bigger again, and reaching stability points according to the model.

But for the Leslie model, the starting values 10 and 0.01 would ensure the extinction of the both predator and prey.

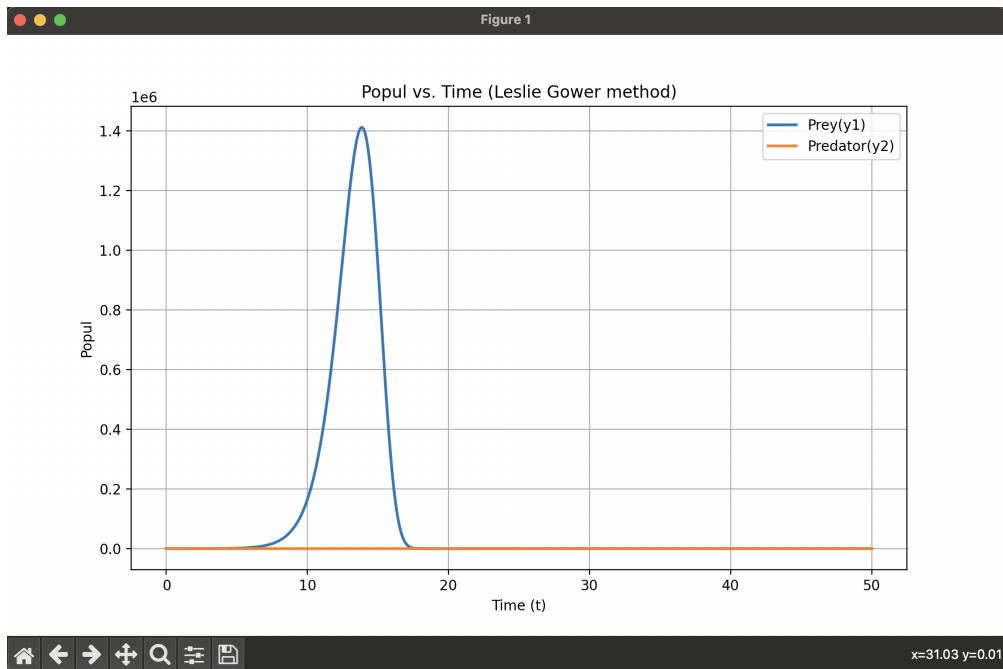


Figure 11: Extinction for Leslie model

1.2 Part 9.2

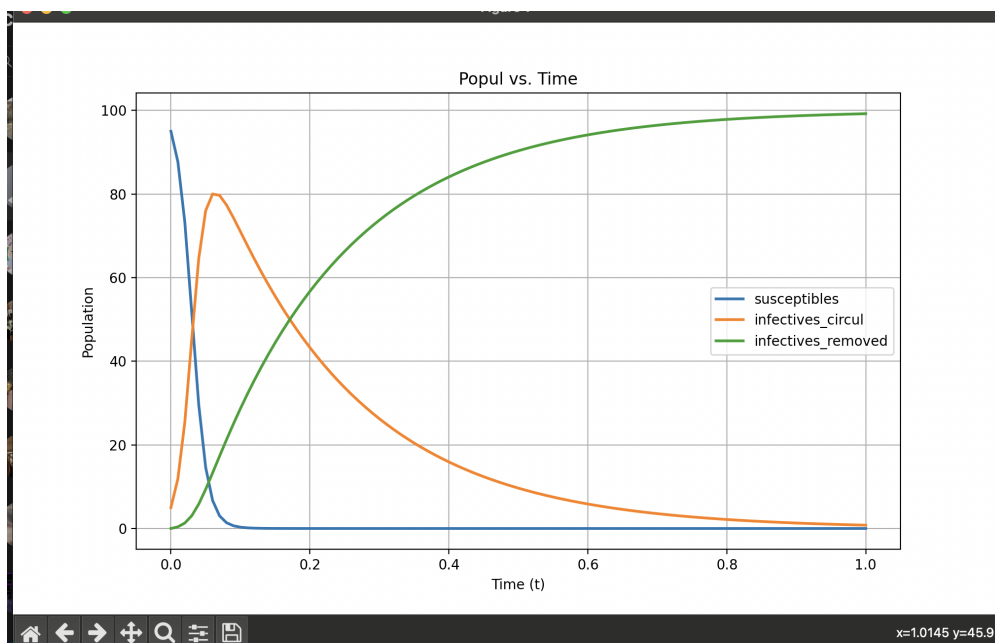


Figure 12: Population vs time 9.2

2 Problem 1.2

2.0.1 Problem 5.23

The following image shows the solutions and the convergence rates of the two methods, custom made newton methods, and library routine methods. We can see that our values are close to the custom routine and our custom made routine converges faster.

```
Comparison of solutions:
Custom Newton's: [0.87796576 0.67675697 1.33085541]
library routine: [0.87796576 0.67675697 1.33085541]

Comparison of iterations:
Custom Newton's: 6
library routine: 14
```

Figure 13: Comparison of the methods

The following is the convergence rates for the Custom Newton's method:

```
Convergence rates for Custom Newton's:
[0.2819832348395953, 0.13464314429745303, 0.028596074710485427, 0.0008019864200292941,
6.28332526650806e-07]
```

Figure 14: Convergence Rates

The convergence rates above include convergence rate at every iteration, so we need to choose the last convergence rate because we are interested in the division of the final two residuals at the last iteration.

2.0.2 Problem 5.24

In the problem 5.24, we can see that there are 2 solutions:

```
5_24 solution
Problem 5_24 solution [x1, x2, w1, w2]: [ 0.57735027 -0.57735027  1.          1.
]
Problem 5_24 iterations: 15
```

Figure 15: Problem 5.24

```
5_24 solution
Problem 5_24 solution [x1, x2, w1, w2]: [-0.57735027  0.57735027  1.          1.
]
Problem 5_24 iterations: 15
```

Figure 16: Problem 5.24 second solution

We can see that in the two solutions the x_1 and x_2 values permute. This is expected as we can see that changing the values of x would result in the same nonlinear equations if we also change w values accordingly. But as w values are same here, it does not matter if we also change w or no. And, to reach those solutions we need to select and try different starting initial guesses for our newton method to converge to those values.

2.0.3 Extra

The following is the example of finding equations for two point Gaussian quadrature. As we know that we are adding the monomials with weight w_i , we can derive the equations for $n = 3$ the same way.

$$\begin{aligned}
w_1 + w_2 &= \int_{-1}^1 1 \, dx = 2, \\
w_1 x_1 + w_2 x_2 &= \int_{-1}^1 x \, dx = 0, \\
w_1 x_1^2 + w_2 x_2^2 &= \int_{-1}^1 x^2 \, dx = \frac{2}{3}, \\
w_1 x_1^3 + w_2 x_2^3 &= \int_{-1}^1 x^3 \, dx = 0.
\end{aligned}$$

Figure 17: Two point Guassian Quadrature

$$\begin{aligned}
w_1 + w_2 + w_3 &= \int_{-1}^1 1 \, dx = 2 \\
w_1 x_1 + w_2 x_2 + w_3 x_3 &= \int_{-1}^1 x \, dx = 0 \\
w_1 x_1^2 + w_2 x_2^2 + w_3 x_3^2 &= \int_{-1}^1 x^2 \, dx = \frac{2}{3} \\
w_1 x_1^3 + w_2 x_2^3 + w_3 x_3^3 &= \int_{-1}^1 x^3 \, dx = 0 \\
w_1 x_1^4 + w_2 x_2^4 + w_3 x_3^4 &= \int_{-1}^1 x^4 \, dx = \frac{2}{5} \\
w_1 x_1^5 + w_2 x_2^5 + w_3 x_3^5 &= \int_{-1}^1 x^5 \, dx = 0
\end{aligned}$$

So we can solve the above system the same we did it for the 5.24 problem.

The following is the calculated solutions for this nonlinear system. Using the same intuition in problem 5.24, we can see that if we permute the values of x and permute w values accordingly, the nonlinear equa-

tions will not change. That is why we can say that there are 6 different answers for extra problem as all three x values are different. We found this value by taking factorial of 3.

```
Extra solution
Extra solution [x1, x2, x3, w1, w2, w3]: [-7.74596669e-01  7.74596669e-01  4.22344169e
-10  5.55555556e-01
 5.55555555e-01  8.88888889e-01]
Extra iterations: 15
(chaco) Nijat@MacBook-Pro:Final HW: nijatabbasov
```

Figure 18: Extra problem solutions

3 Problem 1.3

The followings are the results, residuals and iteration numbers.

```
eWarning: The number of calls to function has reached maxfev = 100.
solution = fsolve(wrap_sistem, initial_guess, xtol=tol, maxfev=max_iter)
(a): Solution = [11.41173833 -0.89682696]
    Final Residual = 6.998893137077413
    Convergence Rate = 1.0000025548550295
    Iterations = 31
(b) Solution [1.00000000e+00 2.45937814e-15 2.00000000e+00]
    Final Residual = 3.552713678800501e-15
    Convergence Rte = 2.50605682609156e-06
    Iterations = 34
(e): Solution = [1.21794979e-05 8.21074187e+00]
    Final Residual = 0.00016183009201861673
    Convergence Rate = 0.843081492693698
    Iterations = 101
```

Figure 19: Results 1.3

For the first one, the starting initial points are away from the real solutions. That is why our method gets stuck at some point and the residual stops decreasing. That is why, we can see that the convergence rate is 1. As the definition of convergence rate is the division of final residual and residual one before that we can see that at the end, our residual stops decreasing and that is why, our residual stays the same and convergence rate becomes zero.

For the second one low convergence rate indicates that the error becomes zero very rapidly. This can also be evident from the Final residual.

For the third problem, we can see that the solution is away from the initial guesses. That is why, it is reasonable that the iterations are big.

4 Problem 1.5

The following is the Principal Submatrix. I chose the matrix in the uppest lefttest side of the matrix T.

```
5x5 Principal Submatrix of T:
[[144.      50.91168825  0.      0.      0.      ]
 [ 50.91168825 144.     -62.35382907  0.      0.      ]
 [  0.      -62.35382907 144.     -65.7267069  0.      ]
 [  0.      0.     -65.7267069 144.     -67.98823428]
 [  0.      0.      0.     -67.98823428 144.     ]]
```

Figure 20: Principal Submatrix

The following is the comparison of the exact and computed eigenvalues, we can see that maximum absolute error is 5 and if we were to find the relative error we would divide 5 by 170, so we can say that the computed eigenvalues are close to the exact values. However the reason for that error is because the 9th and 10th eigenvalues are very close to each other.

```
[ 0.      0.      0.     -67.98823428 144.     ]
/Users/nijatabbasov/Desktop/KAIST_SEM_3/numerical analysis/Final_HW/Pr_5.py:81: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will
error in future. Ensure you extract a single element from your array before performin
g this operation. (Deprecated NumPy 1.25.)
    eigenvalues[i] = solution_newton

Comparison:

Computed Eigenvalues: [268.70778799 242.35973916 242.35382907 216.88110451 208.234232
64
206.35382907 180.51135268 180.      175.05627347 170.35382907]

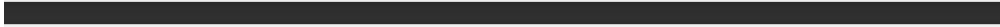
Exact Eigenvalues:   [268.70765814 242.35382907 242.35382907 216.      206.353829
07
206.35382907 180.      180.      170.35382907 170.35382907]
```

Figure 21: Eigenvalue comparison

The following is the text in the book explaining that when the eigenvalues are close to each other, finding the roots of the secular equation

can be inaccurate.

can be verified by direct multiplication. Unfortunately, when two eigenvalues are very close, the cancellation incurred in $\mathbf{D} - \lambda\mathbf{I}$ may cause the resulting eigenvectors to be inaccurate and far from orthogonal, so a more sophisticated approach is



Chapter 4: Eigenvalue Problems

required. These difficulties with numerical stability and eigenvector orthogonality

Figure 22: Eigenvalues close to each other

4.1 Reference

1. scientific-computing-michael-t-heath.pdf